

# Kafka

Distributed message queue – "సందేశాలని ఒక queue lo పెట్టి, చాలా consumers తో లాగేద్దాం" అని అందరూ మొదలుపెడతారు. Disk వేగం గురించి మాట్లాడతారు. అది సమస్య కాదు.

మూడు విరుపులు. మొదటిది – ఒకే queue, 16 consumers అయితే **2,000 lo 106 keys** క్రమం తప్పతాయి. రెండోది – మీరు ఎన్నుకున్న partition సంఖ్య **మూడు వేర్వేరు పనులు** చేస్తుంది, మరియు దాన్ని మార్చితే **55% keys** క్రమం కోల్పోతాయి. మూడోది – **acks=all కూడా** ఆమోదించిన సందేశాలను పోగొడుతుంది.

## ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `lo` run చేసినదే – disk సంఖ్యలతో సహా, అవి ఈ laptop మీద కొలిచినవి.

ఈ **PROBLEM LO** అందరూ చెప్పే ఒక వాక్యం ఉంది, మరియు అది ఇప్పుడు తప్పు

**"Kafka వేగంగా ఉంటుంది ఎందుకంటే అది disk మీద వరుసగా (sequentially) రాస్తుంది."**

ఆ వాక్యం తిరిగే disks (HDD) కాలం నుంచి వచ్చింది, అక్కడ యాదృచ్ఛిక seek కి 10 ms పట్టింది. §1 lo ఈ laptop యొక్క NVMe SSD మీద కొలిచాను: వరుస క్రమం యాదృచ్ఛిక క్రమం కంటే **1.62 రెట్లు** మాత్రమే వేగం.

1.62 అనేది 1000 కాదు. కాబట్టి ఆ వివరణ **ఆధునిక hardware మీద నిలబడదు**.

నిజమైన కారణం వేరే, మరియు అది కూడా కొలిచాను: **945 రెట్లు**. అది §1 lo ఉంది.

## విషయ సూచిక (Table of Contents)

### Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

### Part 2 – మొదటి విరుపు: క్రమం

3. Step – ఒక queue, చాలా consumers
4. మొదటి విరుపు – 2,000 lo 106 keys క్రమం తప్పాయి
5. Step – `hash(key) % partitions`

### Part 3 – రెండో విరుపు: ఆ ఒక్క సంఖ్య

6. Step – partitions ఎన్ని పెట్టాలి?
7. రెండో విరుపు – ఒక సంఖ్య మూడు బాధ్యతలు మోస్తుంది
8. Step – ఎక్కువ partitions, ముందే

### Part 4 – మూడో విరుపు: మన్నిక

9. Step – replication · leader మరియు followers
10. మూడో విరుపు – `acks=all` కూడా పోగొడుతుంది
11. Step – `min.insync.replicas` · మరియు దాని ధర

### Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code · 40 లక్షల సందేశాలు · mutation testing
13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటితో చెప్పాల్సిన English script
15. Follow-ups – exactly-once, compaction, tiered storage
16. ఏమి నేర్చుకున్నాం

## Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

### 1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

సెకనుకి ఒక కోటి సందేశాలు. ఒక్కొక్కటి 1 KB. మూడు కాపీలు. Consumers వాటిని తమ వేగంతో చదవాలి, మరియు ఒకరి వేగం ఇంకొకరిని ఆపకూడదు.

సెకనుకి ఒక కోటి సందేశాలు • ఒక్కో దాని పరిమాణం 1 KB

ముడి bandwidth : 10 GB/s  
3 కాపీలతో : 31 GB/s  
రేజికి నిల్వ (1 కాపీ) : 0.9 PB

ఇప్పుడు అసలు ప్రశ్న: **ఒక machine సెకనుకి ఎన్ని సందేశాలు రాయగలదు?** ఈ laptop మీద కొలిచాను – 200-byte records, ఒక్కో సందర్భం మూడుసార్లు, ఉత్తమమైనది:

200-byte records • ఒక్కో సందర్భం 3 సార్లు, ఉత్తమమైనది

| ఎలా రాస్తున్నాం                | సెకనుకి records | MB/s |
|--------------------------------|-----------------|------|
| ఒక్కొక్కటి • ప్రతిదానికీ fsync | 301             | 0    |
| ఒక్కొక్కటి • fsync లేదు        | 6,89,168        | 138  |
| 100 ముద్ద • ముద్దకి fsync      | 29,866          | 6    |
| 1000 ముద్ద • ముద్దకి fsync     | 2,84,445        | 57   |
| 1000 ముద్ద • fsync లేదు        | 2,01,19,206     | 4024 |

ప్రతిదానికీ fsync → 1000 ముద్దకి fsync : 945× వేగం  
ముద్ద చేయడం ఒక్కోటి (fsync లేకుండా) : 29.2× వేగం

**ఒక్కొక్క సందేశానికీ FSYNC → సెకనుకి 301**

మూడు వందల ఒకటి. ఒక ఆధునిక NVMe SSD మీద.

ఎందుకంటే fsync disk యొక్క bandwidth ని వాడదు – అది disk యొక్క **latency** ని వాడుతుంది. ఎంత పెద్ద disk అయినా, "నిజంగా రాశావా?" అనే ప్రశ్నకి జవాబు రావడానికి కొంత సమయం పడుతుంది, మరియు మీరు ఆ సమయాన్ని **ప్రతి సందేశానికీ** చెల్లిస్తున్నారు.

ఆ ఒక్క తేడా మొత్తం system ని నిర్ణయిస్తుంది:

ఒకే hardware – రాసే విధానాన్ని బట్టి ఎన్ని machines:

| రాసే విధానం                | machine కి సెకనుకి | కావాల్సిన machines |
|----------------------------|--------------------|--------------------|
| ఒక్కొక్క సందేశానికీ fsync  | 301                | 99,668             |
| 1000 ముద్ద • ముద్దకి fsync | 2,84,445           | 106                |

#### 99,668 MACHINES VS 106 MACHINES – ఒకే HARDWARE, ఒకే సందేశాలు

ఇదే ఈ problem యొక్క నిజమైన "storage" పాఠం, మరియు అది "వరుసగా రాయడం" కాదు.

నేను అదే laptop మీద వరుస vs యాదృచ్ఛిక రాతని కూడా కొలిచాను (128 MB, ముందే కేటాయించిన file, 3 runs lo ఉత్తమమైనది):

వరుస క్రమంలో : 2138 MB/s • యాదృచ్ఛిక క్రమంలో : 1319 MB/s → **1.62×**

**1.62 రెట్లు vs 945 రెట్లు.** కాబట్టి interview lo "sequential disk" అని చెప్పకండి – "**batching మరియు fsync ని amortize చేయడం**" అని చెప్పండి. అది ఈ దశాబ్దపు జవాబు.

అంటే throughput ఒక సమస్య కాదు – **106 machines**. మిగతా doc అసలు కష్టమైన మూడు ప్రశ్నల గురించి: **క్రమం, సమాంతరత, మరియు మన్నిక.**

## 2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

| ప్రశ్న                                          | ఎందుకు అడుగుతున్నాం                                         |
|-------------------------------------------------|-------------------------------------------------------------|
| క్రమం అవసరమా – అయితే దేని ప్రకారం?              | "అన్నిటికీ" అంటే partitions ఒక్కటే. "ఒక్కో user కి" అంటే §5 |
| ఒక సందేశం రెండుసార్లు వస్తే ఫరవాలేదా?           | ఫరవాలేదు కాదంటే §15 – మరియు జవాబు మీరు అనుకున్నది కాదు      |
| ఒక సందేశం పోతే?                                 | §10. acks విలువ ఈ ఒక్క జవాబు మీద ఆధారపడుతుంది               |
| ఎంతమంది వేర్వేరు consumers, వేర్వేరు వేగాలతో?   | ఇదే queue కి బదులు log ఎందుకు అనేదానికి కారణం               |
| సందేశాలని ఎంతకాలం ఉంచాలి?                       | గంటలా, వారాలా, శాశ్వతమా – నిల్వ లెక్క పూర్తిగా మారుతుంది    |
| Consumer వెనకబడితే ఏమి కావాలి?                  | ఆలస్యమా, లేక సందేశాలు వదిలేయడమా – ఇది product నిర్ణయం       |
| Keys ఎలా పంచబడ్డాయి – సమానంగానా, కొన్ని వేడివా? | వేడి keys ఉంటే §7 యొక్క ceiling ఇంకా తక్కువ                 |
| Consumers ఎంత తరచుగా చేరతారు/పోతారు?            | ప్రతిసారీ rebalance – §12 lo దాని ఖర్చు                     |

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

**"క్రమం ఎవరికి కావాలి – మొత్తం topic కా, లేక ఒక్క key కా?"**

ఎందుకంటే ఈ రెండూ పూర్తిగా వేర్వేరు systems. **మొత్తం topic కి క్రమం** అంటే partitions ఒక్కటే, అంటే సమాంతరత ఒక్కటే – మీ మొత్తం throughput ఒక్క machine కి పరిమితం.

**ఒక్క key కి క్రమం** అంటే మీరు వేల partitions వాడొచ్చు. మరియు §7 lo కొలుస్తాం – ఆ partition సంఖ్య తర్వాత మార్పడం చాలా ఖరీదైనది.

99% సందర్భాల్లో సరైన జవాబు "ఒక్క key కి" – కానీ అది **అడిగి తెలుసుకోవాలి**, ఊహించకూడదు.

## Part 2 – మొదటి విరుపు: క్రమం

### 3. Step – ఒక queue, చాలా consumers

సహజమైన మొదలు: ఒక queue, చాలామంది consumers. ఎవరు ఖాళీగా ఉంటే వాళ్ళు తర్వాతి సందేశాన్ని తీసుకుంటారు.

[m1] [m2] [m3] [m4] [m5] ... → consumer A, B, C, D ఎవరైనా

ఇది చక్కగా సమతుల్యం అవుతుంది – నెమ్మదైన consumer తక్కువ తీసుకుంటుంది, వేగమైనది ఎక్కువ. ఇది **ఖచ్చితంగా సరైన load balancing**.

ఇప్పుడు సందేశాలకి ఒక అర్థం ఇద్దాం. ప్రతి సందేశానికి ఒక **key** – ఒక account id అనుకోండి – మరియు ఆ account కి సంబంధించిన సందేశాలు వరుసగా వస్తాయి:

key 42: "₹100 జమ" → "₹500 తీసివేత" → "నిల్వ చూపించు"

ఆ మూడూ ఆ క్రమంలోనే ప్రాసెస్ కావాలి.

### 4. మొదటి విరుపు – 2,000 to 106 keys క్రమం తప్పాయి

50,000 సందేశాలు, 2,000 keys, ప్రాసెసింగ్ సమయం ఒక్కే సందేశానికే వేరు (1-20 units), 10 seeds:

ఒకే queue • ఏ consumer అయినా ఏ సందేశాన్నైనా • 50,000 సందేశాలు, 2,000 keys

| consumers | క్రమం తప్పిన సందేశాలు | సందేశాల్లో % | దెబ్బతిన్న keys | keys to % |
|-----------|-----------------------|--------------|-----------------|-----------|
| 1         | 0                     | 0.0%         | 0               | 0.0%      |
| 2         | 148                   | 0.0%         | 7               | 0.4%      |
| 4         | 394                   | 0.1%         | 20              | 1.0%      |
| 8         | 996                   | 0.2%         | 49              | 2.5%      |
| 16        | 2,174                 | 0.4%         | 106             | 5.3%      |

### ఏ నిలువు వరుస చూడాలో అనేదే ఇక్కడి పాఠం

"సందేశాల్లో 0.4%" అనే సంఖ్య **చిన్నదిగా** అనిపిస్తుంది. Interview lo ఎవరైనా "అది సరిపోతుందిలే" అనొచ్చు.

కానీ మీ user ఒక సందేశం కాదు – ఒక **account**. మరియు 2,000 accounts lo **106 accounts** (5.3%) కనీసం ఒక్కసారైనా తప్పు క్రమంలో ప్రాసెస్ అయ్యాయి.

**"నిల్వ చూపించు" ఆ ₹500 తీసివేత కంటే ముందు నడిస్తే user కి తప్పు నిల్వ కనిపిస్తుంది.**

సందేశాల శాతం ఒక engineering సంఖ్య. Keys శాతం ఒక **వ్యాపార** సంఖ్య. రెండేది చెప్పండి.

మరియు వరుస 1 ని గమనించండి: **ఒక్క consumer → 0 ఉల్లంఘనలు**. అంటే క్రమం మరియు సమాంతరత మధ్య ఇది ఒక నేరుగా ఉన్న వ్యాపారం – consumers పెంచిన కొద్దీ ఉల్లంఘనలు పెరుగుతాయి.

## 5. Step – hash(key) % partitions

పరిష్కారం: queue ని **partitions** గా విభజించడం, మరియు ఒక key ఎప్పుడూ **ఒకే partition** కి వెళ్ళడం.

```
partitionOf(key) { return hash(key) % this.partitions; }
```

ఆపై రెండో సగం, మరియు ఇది అంతే ముఖ్యం: **ఒక partition ని ఒక group lo ఒకే ఒక consumer చదువుతుంది.**

```
partition 0: [k42-a] [k42-b] [k7-a] → consumer A ఒక్కడే
partition 1: [k13-a] [k13-b] → consumer B ఒక్కడే
partition 2: [k99-a] [k42-c]? ← కాదు. k42 ఎప్పుడూ partition 0 కే.
```

ఆ రెండూ కలిపితే: ఒక key యొక్క అన్ని సందేశాలూ **ఒకే partition lo, ఒకే consumer చేతిలో, వరుసగా.**

hash(key) % partitions ప్రకారం విభజిస్తే · 16 consumers

partitions | క్రమం తప్పిన సందేశాలు | దెబ్బతిన్న keys

|    |  |   |  |   |
|----|--|---|--|---|
| 1  |  | 0 |  | 0 |
| 4  |  | 0 |  | 0 |
| 16 |  | 0 |  | 0 |
| 64 |  | 0 |  | 0 |

**KAFKA** ఇచ్చే హామీ ఖచ్చితంగా ఏమిటి

**"క్రమం" అనేది topic కి కాదు – ఒక్క partition కి.**

మరియు మీరు key ని ఎన్నుకోవడం ద్వారా **దేనికి క్రమం కావాలో మీరే నిర్ణయిస్తారు**. Account id ని key గా పెడితే ఒక్క account కి క్రమం. దేశం పేరు పెడితే ఒక్క దేశానికి క్రమం – మరియు భారతదేశం అనే ఒక్క partition కుంగిపోతుంది.

**Partition key అనేది ఒక performance నిర్ణయం కాదు. అది మీ correctness నిర్వచనం.**

# Part 3 – రెండో విరుపు: ఆ ఒక్క సంఖ్య

## 6. Step – partitions ఎన్ని పెట్టాలి?

ఇప్పుడు ఒక సంఖ్య ఎన్నుకోవాలి. "16 సరిపోతుందిలే, తర్వాత కావాలంటే పెంచుకోవచ్చు" అనేది సహజమైన ఆలోచన.

ఆ రెండు భాగాలూ ప్రమాదకరమైనవి, మరియు మూడు కొలతలతో చూపిస్తాను.

## 7. రెండో విరుపు – ఒక సంఖ్య మూడు బాధ్యతలు మోస్తుంది

బాధ్యత 1 – సమాంతరత యొక్క పైకప్పు. 16 partitions, consumers పెంచుతూ:

1. consumers పెంచితే throughput పెరుగుతుందా? • 16 partitions • 50,000 సందేశాలు

| consumers | పనిచేసేవి | ఖాళీగా | పూర్తయిన సమయం | వేగం   |
|-----------|-----------|--------|---------------|--------|
| 1         | 1         | 0      | 5,22,702      | 1.00×  |
| 2         | 2         | 0      | 2,61,656      | 2.00×  |
| 4         | 4         | 0      | 1,31,445      | 3.98×  |
| 8         | 8         | 0      | 66,373        | 7.88×  |
| 16        | 16        | 0      | 36,359        | 14.38× |
| 32        | 16        | 16     | 36,359        | 14.38× |
| 64        | 16        | 48     | 36,359        | 14.38× |

**64 CONSUMERS • 48 ఖాళీగా • వేగం ఒక్క దశాంశం కూడా పెరగలేదు**

16 వరకు దాదాపు సరళంగా పెరుగుతుంది (14.38×, 16 కాదు – ఎందుకంటే partitions సరిగ్గా సమానంగా నిండవు).

ఆ తర్వాత పూర్తిగా చదును. Consumers కలపడం ఖాళీ consumers ని కలపడమే.

మీ గరిష్ట consumer సమాంతరత = partitions సంఖ్య. ఆ సంఖ్యని మీరు మొదటి రోజే ఎన్నుకున్నారు.

బాధ్యత 2 – key → partition చిత్రం. ఇప్పుడు partitions పెంచుదాం:

2. partitions పెంచితే ఎన్ని keys వేరే partition కి మారతాయి?

| నుంచి → కి | మారిన keys | %     |
|------------|------------|-------|
| 16 → 17    | 94,065     | 94.1% |
| 16 → 24    | 66,682     | 66.7% |
| 16 → 32    | 50,064     | 50.1% |

16 → 64 | 75,145 | 75.1%  
32 → 64 | 50,193 | 50.2%

"రెట్టింపు చేస్తే నయం" అనేది కూడా సగం **KEYS** ని కదిలిస్తుంది

16 → 17: **94.1%**. ఇది ఊహించదగినదే - % N అనేది N మారితే ప్రతిదీ మారుస్తుంది.

కానీ 16 → 32 కూడా **50.1%**. "రెట్టింపు చేయడం మంచిది" అనే సలహా సగం **నష్టాన్ని** మాత్రమే తగ్గిస్తుంది, నివారించదు.

**బాధ్యత 3 – ఆ కదలిక జరిగే క్షణంలో క్రమం ఏమవుతుంది.** ఇక్కడే అసలు నష్టం:

3. ఆ మార్పు జరుగుతున్నప్పుడు క్రమం ఏమవుతుంది?

మార్పు క్షణంలో పాత partitions lo చదవనివి : 4,000 సందేశాలు  
క్రమం తప్పిన సందేశాలు : 6,743  
కనీసం ఒక్కసారి దెబ్బతిన్న keys : 2,757 / 5,000 (55.1%)

**55.1% KEYS – §4 LO మనం పరిష్కరించిన సమస్య తిరిగి వచ్చింది, పెద్దదిగా**

కారణం సులభమైనది: partitions పెంచిన క్షణంలో, key 42 యొక్క పాత సందేశాలు **ఇంకా partition 10 lo చదవకుండా** ఉన్నాయి, మరియు కొత్త సందేశాలు **ఇప్పటికే partition 26 కి** వెళ్తున్నాయి.

రెండు partitions స్వతంత్రంగా చదవబడతాయి. కాబట్టి కొత్తవి పాతవాటి కంటే **ముందే** ప్రాసెస్ కావచ్చు.

§4 lo 5.3% keys దెబ్బతిన్నాయి. ఇక్కడ **55.1%**.

**కాబట్టి ఆ ఒక్క సంఖ్య ఏమి నిర్ణయిస్తుంది**

1. మీ గరిష్ట consumer సమాంతరత (పట్టిక 1)
2. ఏ key ఎక్కడ ఉంటుంది (పట్టిక 2)
3. మరియు అందువల్ల – **దాన్ని మార్చడం మీ ఏకైక correctness హామీని విరుస్తుంది** (పట్టిక 3)

మూడో దాని వల్ల మొదటి దాన్ని సరిదిద్దలేరు. **Partition సంఖ్య ఆచరణలో శాశ్వతమైనది.**

## 8. Step – ఎక్కువ partitions, ముందే

కాబట్టి నియమం సులభమైనది మరియు కొంచెం అసౌకర్యంగా ఉంటుంది: **మీకు ఇప్పుడు అవసరమైన దాని కంటే ఎక్కువ partitions తో మొదలుపెట్టండి.**

కావాల్సింది 16 అయితే 128 పెట్టండి. అదనపు partitions ఖాళీగా కూర్చోవు – అవి కేవలం **చిన్నవిగా** ఉంటాయి, మరియు ఒక consumer చాలా partitions ని ఏకకాలంలో చదవగలదు.

### ఎక్కువ PARTITIONS ఉచితం కాదు – ఖర్చు ఏమిటి

ప్రతి partition ఒక file, ఒక leader, ఒక ISR జాబితా, మరియు ప్రతి producer కి ఒక buffer. కాబట్టి:

**Producer వైపు** – ఎక్కువ partitions అంటే ఒక్కో partition కి చిన్న ముద్దలు, అంటే **₹1 యొక్క 945x తగ్గుతుంది**. ఇదే నిజమైన పరిమితి.

**Broker వైపు** – ఎక్కువ open files, ఎక్కువ metadata, leader ఎన్నికకి ఎక్కువ సమయం.

కాబట్టి "లక్ష partitions" జవాబు కాదు. జవాబు: **మీ 3-5 ఏళ్ళ గరిష్ట consumer సంఖ్య × 2**, మరియు ఆ సంఖ్యని ఒక నిర్ణయంగా రాసి పెట్టడం.

మరియు ఒక ముఖ్యమైన మినహాయింపు – **మార్చాల్సిన వస్త్ర ఏమి చేయాలి**: partitions పెంచడం కాదు, **కొత్త topic** తయారు చేసి, పాతది పూర్తిగా ఖాళీ అయ్యాక consumers ని మార్చడం. ఇది నెమ్మదైనది మరియు విసుగు తెప్పించేది, మరియు **₹7 యొక్క 55.1% ని అది నివారిస్తుంది**.

# Part 4 – మూడో విరుపు: మన్నిక

## 9. Step – replication • leader మరియు followers

ప్రతి partition కి మూడు కాపీలు: ఒక **leader** (రాతలు, చదవడాలు అక్కడే) మరియు రెండు **followers** (leader నుంచి కాపీ చేసుకుంటారు).

Producer ఎప్పుడు "సరే, రాశాం" అని వినాలి? రెండు ఎంపికలు:

acks=1 → leader రాసిన వెంటనే. వేగం ఎక్కువ.  
acks=all → ప్రస్తుత ISR lo అందరికీ చేరాక. సురక్షితం.

**ISR** = *in-sync replicas* – leader తో "తగినంత దగ్గరగా" ఉన్న కాపీలు. ఒక follower చాలా వెనకబడితే (GC pause, disk stall) అది **ISR నుంచి తొలగించబడుతుంది**, మరియు తిరిగి పట్టుకున్నాక చేరుతుంది.

ఇప్పుడు simulate చేద్దాం: leader అప్పుడప్పుడూ చనిపోతుంది, followers అప్పుడప్పుడూ ఆగిపోతారు. Leader చనిపోతే ISR lo అత్యధిక offset ఉన్నవాడు కొత్త leader.

**కొలిచేది ఒక్కటే: producer కి "సరే" అని చెప్పి, ఆ తర్వాత పోగొట్టిన సందేశాలు.**

## 10. మూడో విరుపు – acks=all కూడా పోగొడుతుంది

1 leader + 2 followers • 40 runs × 20,000 ticks

leader మరణాలు: 609 • ISR కుంచిచున్న సార్లు: 2,577

ISR lo leader ఒక్కడే ఉన్న ticks: 2,43,538 / 8,00,000 (30.4%)

| ఏర్పాటు                 | ఆమోదించినవి | ఆమోదించి పోగొట్టినవి | నష్టం % |
|-------------------------|-------------|----------------------|---------|
| acks=1                  | 70,95,029   | 15,389               | 0.217%  |
| acks=all • min.insync=1 | 70,78,884   | 190                  | 0.003%  |
| acks=all • min.insync=2 | 52,58,146   | 0                    | 0.000%  |

మొదటి వరుస ఊహించినదే: **acks=1 → 15,389 సందేశాలు ఆమోదించి పోగొట్టాం**. Leader "సరే" అన్నాక, followers కాపీ చేసుకునే లోపు అది చనిపోయింది.

రెండో వరుస ఊహించనిది:

ACKS=ALL • ఇంకా 190 సందేశాలు పోయాయి

"అందరికీ చేరాకే సరే" అన్నప్పుడు నష్టం సున్నా కావాలి. కాదు.

ఎందుకంటే "అందరూ" అంటే "ప్రస్తుత ISR లో అందరూ". మరియు పైన ఉన్న సంఖ్య చూడండి:

**30.4% ticks లో ISR లో leader ఒక్కడే ఉన్నాడు.**

ఆ క్షణాల్లో `acks=all` అంటే "leader కి చేరాక" – అంటే సరిగ్గా `acks=1`. System మీకు చెప్పదు. అది **మౌనంగా** బలహీనపడుతుంది.

ఆ వివరణ నిజమని నిరూపించాలి, కాబట్టి ఒక control నడిపాను – **followers ఎప్పుడూ ఆగకుండా**, అంటే ISR ఎప్పుడూ కుంచించుకోకుండా:

control – followers ఎప్పుడూ ఆగకపోతే • 40 runs

ISR కుంచించుకున్న సార్లు : 0

leader మరణాలు : 625

`acks=all` • `min.insync=1` → ఆమోదించినవి 73,51,147 • పోగొట్టినవి 0

**625 leader మరణాలు, సున్నా నష్టం.** కాబట్టి ఆ 190 ఖచ్చితంగా ISR కుంచించుకున్న కిటికీ నుంచే వచ్చాయి – ఊహా కాదు.

## 11. Step – `min.insync.replicas` • మరియు దాని ధర

పరిష్కారం ఒక రెండో setting, మరియు ఈ రెండూ కలిపి మాత్రమే అర్థవంతమైనవి:

`acks=all + min.insync.replicas=2`

అర్థం: "ISR లో కనీసం రెండు కాపీలు లేకపోతే **రాతని తిరస్కరించు**." మూడో వరుసలో ఫలితం – **సున్నా నష్టం**.

కానీ దానికి ఒక ధర ఉంది, మరియు అదే ఈ section:

అదే ఏర్పాట్లు – అభ్యుత్తరత (availability) వైపు నుంచి

| ఏర్పాటు                              | ఆమోదించినవి | తిరస్కరించినవి | తిరస్కరణ % |
|--------------------------------------|-------------|----------------|------------|
| <code>acks=1</code>                  | 70,95,029   | 0              | 0.0%       |
| <code>acks=all • min.insync=1</code> | 70,78,884   | 0              | 0.0%       |
| <code>acks=all • min.insync=2</code> | 52,58,146   | 19,07,493      | 26.6%      |

### 26.6% రాతలు తిరస్కరించబడ్డాయి - ఇది BUG కాదు, ఇదే మొత్తం ఉద్దేశం

`min.insync=2` నష్టాన్ని 190 నుంచి 0 కి తెచ్చింది, మరియు నాలుగోవంతు రాతలను తిరస్కరించింది.

ఇది CAP theorem ఒక పుస్తకంలో కాదు, ఒక పట్టికలో. ISR కుంచిచుకున్నప్పుడు మీరు ఒకటే ఎన్నుకోగలరు:

రాతని స్వీకరించి బహుశా పోగొట్టడం (`min.insync=1`), లేదా రాతని తిరస్కరించడం (`min.insync=2`).

రెండూ ఒకేసారి దొరకవు, మరియు ఏ setting కూడా ఆ ఎంపికని తప్పించదు. అది మీ product నిర్ణయం - ఒక చెల్లింపా, ఒక click లెక్కా అనేదాన్ని బట్టి.

### ఒక ఆచరణాత్మక పరిణామం, మరియు ఇది తరచుగా తప్పిపోతుంది

`min.insync.replicas=2` కి మూడు కాపీలు కావాలి - రెండు కాదు.

రెండు కాపీలతో `min.insync=2` పెడితే, ఒక్క follower అగిన క్షణంలో **topic పూర్తిగా రాయలేనిదిగా** మారుతుంది. మూడో కాపీ "అదనపు మన్నిక" కోసం కాదు - **ఒకటి పోయినా రాస్తూ ఉండటం** కోసం.

కాబట్టి `replication.factor=3, min.insync.replicas=2, acks=all` - ఈ మూడూ ఒకే నిర్ణయం, మూడు వేర్వేరు settings కాదు.

# Part 5 – నిరూపణ మరియు ప్రదర్శన

## 12. మొత్తం code · 40 లక్షల సందేశాలు · mutation testing

```
class Broker {
  #parts; #committed = new Map();          // group → partition వారీగా commit చేసిన
  offset

  // §5 – ఒకే key ఎప్పుడూ ఒకే partition కి. ఇదే క్రమానికి ఎక్సెక్ హామీ.
  partitionOf(key) { return hash(key) % this.partitions; }

  produce(key, val) {
    const p = this.partitionOf(key);
    const off = this.#parts[p].length;
    this.#parts[p].push({ key, val, off });
    return { partition: p, offset: off };
  }
  commit(g, p, off) {
    const a = this.#committed.get(g);
    if (off > a[p]) a[p] = off;              // వెనక్కి వెళ్ళకూడదు
  }
}

class Group {
  assign(consumerIds) {                      // rebalance
    for (let p = 0; p < this.b.partitions; p++) {
      this.owner[p] = consumerIds[p % consumerIds.length];
      // rebalance తర్వాత commit చేసిన చోటి నుంచే మళ్ళీ – మధ్యలోనివి తిరిగి వస్తాయి
      this.cursor[p] = this.b.committedOf(this.name, p);
    }
  }
  poll(consumerId, max) { /* తన partitions నుంచి మాత్రమే */ }
  commit(p) { this.b.commit(this.name, p, this.cursor[p]); } // ప్రాసెస్ *తర్వాత*
}
```

400 యాదృచ్ఛిక runs – 2-16 partitions, 1-4 consumers, యాదృచ్ఛిక rebalances:

400 యాదృచ్ఛిక runs · 39,99,118 సందేశాలు · 16,066 rebalances  
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

|            |           |
|------------|-----------|
| produced   | 39,99,118 |
| delivered  | 40,78,953 |
| dupes      | 84,711    |
| commits    | 17,39,295 |
| rebalances | 16,066    |
| polls      | 16,00,000 |

#### మొదట నా నియమం తప్పు, CODE కాదు - 400/400 విఫలం

నా మొదటి నియమం: "ఒకే key యొక్క సందేశాలు ఎప్పుడూ ఉత్పత్తి క్రమంలోనే రావాలి." ఇది 400 runs lo 400 విఫలమైంది.

కారణం: rebalance తర్వాత consumer **commit చేసిన చోటి నుంచి** మళ్ళీ మొదలుపెడుతుంది, కాబట్టి seq 38 ఇప్పటికే చూసిన seq 39 తర్వాత మళ్ళీ వస్తుంది. **అది bug కాదు - అదే at-least-once.**

సరైన నియమం: **\*మొదటి\* delivery క్రమం** ఉత్పత్తి క్రమాన్ని పాటించాలి. పునరావృత్తులు ఆ నియమానికి బయట.

**Kafka క్రమానికి హామీ ఇస్తుంది, ఒక్కసారే చేరుతుందని కాదు. నా నియమం ఆ రెండింటినీ కలిపేసింది.**

ఎనిమిది నియమాలు: **మొదటి-delivery క్రమం, ఒకే partition కి ఒకే యజమాని, commit అయినది తిరిగి రాకూడదు, ప్రతి సందేశం కనీసం ఒక్కసారి, చివరికి lag సున్నా, partitions consumers మధ్య సమానంగా, keys partitions మధ్య సమానంగా, ఒకే key ఒకే partition lo.**

==== mutation testing ====

మార్పు లేని code → 0/400 విఫలం  
key ప్రకారం కాకుండా round-robin partition → 150/150 విఫలం  
ఉదా: key 15: seq 3 మొదటిసారి 4 తర్వాత  
ఒక్కే partition ని అందరికీ ఇస్తే → 115/150 విఫలం  
ఉదా: consumer 0 కి 12 partitions (గరిష్టం 4)  
rebalance తర్వాత cursor ని 0 కి కాకుండా అలాగే ఉంచితే → 150/150 విఫలం  
ఉదా: partition 0: offset 0 తిరిగి వచ్చింది (commit 86)  
commit వెనక్కి వెళ్ళే రక్షణ తీసేస్తే → 0/150 విఫలం  
నేను మొదట రాసిన hash ( $2^{53}$  దాటి ఖచ్చితత్వం పోయేది) → 40/150 విఫలం  
ఉదా: hash అసమతుల్యం: గరిష్ట partition 1360, సగటు 333  
commit ని ప్రాసెస్ \*ముందు\* చేస్తే (at-most-once) → 150/150 విఫలం  
ఉదా: partition 0: offset 8 తిరిగి వచ్చింది (commit 9)

#### ఆ HASH మార్పు నా సొంత తప్పు - మరియు అది ఎంత చెడ్డదో చూడండి

§7 కోసం code రాస్తున్నప్పుడు నేను  $k * 2654435761$  అని రాశాను, `Math.imul` కాదు. JavaScript lo ఆ గుణకారం  $2^{53}$  దాటుతుంది, కాబట్టి ఖచ్చితత్వం పోతుంది.

ఫలితం - 40,000 keys ని 16 partitions lo పంచితే:

సరైనది : ఒక్కే partition కి 2421-2622

నేను రాసినది : ఒక్కే partition కి 2-39673

**40,000 lo 39,673 (99.2%) ఒకే partition మీద.** 64 partitions తో 9 partitions పూర్తిగా ఖాళీ.

దీన్ని నేను కనుగొన్నది "16 → 32 కేవలం 1.4% keys ని కదిలించింది" అనే సంఖ్య **నమ్మశక్యంగా లేకపోవడం** వల్ల. సంఖ్యలు రాయకపోతే ఈ bug నిశ్శబ్దంగా ఉండిపోయేది.

**ఒక మార్పు బతికింది, మరియు దాని కారణం గత doc కంటే వేరు:**

బతికిన మార్పుకి మూడు అర్థాలు ఉండొచ్చు – ఇది మూడోది

if (off > a[p]) అనే రక్షణని తీసేస్తే **0/150** విఫలం. లెక్కపెట్టాను: 17,40,831 commit పీలుపుల్లో **ఒక్కటే** వెనక్కి వెళ్ళలేదు.

(1) పరీక్షలు బలహీనమైనవి – కాదు, మిగతా అయిదు మార్పులూ పట్టుబడ్డాయి.

(2) ఆ పంక్తి చచ్చినది – HLD 12 lo ఇదే జరిగింది, మరియు అక్కడ నేను దాన్ని తీసేశాను.

(3) నా harness ఆ పరిస్థితిని సృష్టించలేదు – ఇక్కడ ఇదే.

వెనక్కి వెళ్ళే commit రావాలంటే ఒక **zombie consumer** కావాలి – rebalance జరిగిందని తెలియని, పాత partitions ని ఇంకా చదువుతున్న consumer. నా harness lo ప్రతి consumer ఒకే Group ని పంచుకుంటుంది, కాబట్టి zombie ఉండటం సాధ్యం కాదు.

కాబట్టి ఆ పంక్తిని ఉంచుతున్నాను, మరియు దాన్ని నేను నిరూపించలేదని చెబుతున్నాను. నిజమైన Kafka దీన్ని generation id తో పరిష్కరిస్తుంది – LLD Deep 25 lo fencing tokens తో కొలిచిన అదే నమూనా.

## 13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

| నిమిషాలు | ఏమి చెప్పాలి                                                   | ఏ సంఖ్య చెప్పాలి                                                          |
|----------|----------------------------------------------------------------|---------------------------------------------------------------------------|
| 0-5      | Requirements + స్కోప్                                          | సెకనుకి 1 కోటి · 1 KB · 3 కాపీలు                                          |
| 5-10     | <b>Throughput సమస్య కాదు – కానీ కారణం sequential disk కాదు</b> | 1.62x vs <b>945x</b> · 99,668 vs 106 machines                             |
| 10-17    | ఒకే queue → క్రమం విరుగుతుంది                                  | 0.4% సందేశాలు కానీ <b>5.3% keys</b>                                       |
| 17-22    | hash(key) % partitions + ఒక partition, ఒక consumer             | అన్ని partition counts lo <b>0</b>                                        |
| 22-32    | <b>ఆ ఒక్క సంఖ్య మూడు పనులు</b>                                 | 64 consumers → 48 ఖాళీ · 16→32 <b>50.1%</b> కదులుతాయి · <b>55.1% keys</b> |
| 32-40    | acks · ISR · <b>acks=all కూడా పోగొడుతుంది</b>                  | ISR=1 అయిన ticks <b>30.4%</b> · 190 పోయాయి                                |
| 40-45    | min.insync=2 మరియు దాని ధర                                     | నష్టం 0, కానీ <b>26.6% తిరస్కరణ</b>                                       |

ఈ **PROBLEM LO** మమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "durability కి `acks=all` పెడతాను" అని చెప్పి ముందుకి వెళ్ళారు.

మీరు ఒక అడుగు ముందుకి: "`acks=all` ఒక్కటే సరిపోదు – నేను కొలిచినప్పుడు అది ఇప్పటికీ 190 ఆమోదిత సందేశాలను పోగొట్టింది. ఎందుకంటే 'all' అంటే 'ప్రస్తుత ISR', మరియు నా simulation lo 30% సమయం ISR lo leader ఒక్కడే ఉన్నాడు. `min.insync.replicas=2` దాన్ని సున్నాకి తెచ్చింది – మరియు 26.6% రాతలను తిరస్కరించింది. ఆ తిరస్కరణ రేటే అసలు నిర్ణయం, మరియు అది product నిర్ణయం."

ఇది ఒక setting కి రెండో setting ఉందని తెలుసని చూపదు – ఒక setting ఒంటరిగా అబద్ధం చెబుతుందని మీరు చూశారని చూపుతుంది.

## 14. నేటికే చెప్పాల్సిన English script

### Throughput గురించి:

"Before I design anything I want to check whether throughput is the hard part. I benchmarked this on my own machine: writing records one at a time with an fsync each gives three hundred records a second on an NVMe SSD – because fsync spends latency, not bandwidth. Batching a thousand records per fsync gives two hundred eighty-four thousand. That's nine hundred forty-five times, and it's the difference between a hundred thousand machines and a hundred and six. I'd also push back on the usual explanation: I measured sequential versus random writes on the same disk and it was only one point six times. The sequential-disk story is from the spinning-disk era. The real answer is batching."

### Ordering గురించి:

"With one queue and sixteen consumers I measured ordering violations on zero point four percent of messages – which sounds fine until you change the denominator. Five point three percent of keys had at least one out-of-order event. If the key is an account, that's one in twenty accounts seeing a stale balance. So I'd partition by key hash, and give each partition exactly one consumer in a group. That takes violations to zero. The important framing is that the partition key isn't a performance choice – it's the definition of what 'in order' means for this system."

### Partition count గురించి:

"Then the part I'd want to spend the most time on. That partition number does three jobs at once. It caps consumer parallelism – I measured sixteen partitions scaling to fourteen times and then completely flat, with forty-eight idle consumers at sixty-four. It decides which key lives where. And because of that, changing it breaks the only guarantee you have: going from sixteen to thirty-two partitions moves half the keys, and in my simulation of the transition, fifty-five percent of keys saw out-of-order messages. So over-partition up front, and if you truly must change it, make a new topic and drain the old one rather than resizing."

### Durability గురించి:

"For durability, acks=1 lost about zero point two percent of acknowledged messages when leaders crashed. I expected acks=all to take that to zero. It didn't – it still lost a hundred ninety, because 'all' means 'all of the current in-sync replicas', and thirty percent of the time in my run the in-sync set was just the leader. So acks=all had silently degraded to acks=1. I confirmed that by running a control with no follower stalls: six hundred twenty-five leader crashes, zero loss. The fix is min.insync.replicas=2, which took loss to zero and rejected twenty-six percent of writes. That rejection rate is the actual decision, and it needs three replicas, not two – otherwise one slow follower makes the topic unwritable."

## 15. Follow-ups – exactly-once, compaction, tiered storage

### "Exactly-once సాధ్యమా?"

చేరడం (delivery) exactly-once కాదు – అది ఎప్పటికీ సాధ్యం కాదు, ఎందుకంటే ack పోతే producer మళ్ళీ పంపాల్సిందే. సాధ్యమైనది exactly-once **ప్రభావం**, మరియు దానికి రెండు ముక్కలు: producer వైపు ఒక **sequence number** (broker నకలుని గుర్తించి పడేస్తుంది), మరియు consumer వైపు **ప్రాసెసింగ్ మరియు offset commit ఒకే transaction లో** ఉండటం. నా harness లో 40 లక్షల సందేశాలకి **84,711 నకళ్ళు** వచ్చాయి – అంటే at-least-once అనేది ఒక సిద్ధాంతం కాదు, ఒక రోజువారీ వాస్తవం. HLD Deep 09 లో ఇదే idempotency కొలిచాను.

### "పాత సందేశాలని ఏమి చేయాలి?"

రెండు విధానాలు, మరియు ఎంపిక **topic ఏమి సూచిస్తుందో** దాని మీద ఆధారపడుతుంది. Topic ఒక **సంఘటనల ప్రవాహం** అయితే (clicks, logs) – కాలం ప్రకారం తొలగించండి. Topic ఒక **ప్రస్తుత స్థితి** అయితే (ఒక్కో user యొక్క తాజా profile) – **compaction**: ఒక్కో key కి చివరి విలువ మాత్రమే ఉంచండి. అప్పుడు topic ఒక log కాదు, ఒక **table** – మరియు కొత్త consumer మొదటి నుంచి చదివి పూర్తి స్థితిని పునర్నిర్మించగలదు.

### "రోజుకి 0.9 PB – అంతా brokers మీదేనా?"

కాదు, మరియు ఇక్కడే **tiered storage**: తాజా segments (గంటలు) brokers యొక్క local disk మీద, పాతవి object storage (S3) కి. Consumers 99% సమయం తాజా భాగాన్నే చదువుతారు. దీని పెద్ద లాభం నిల్వ ఖర్చు కాదు – **broker ని మార్చినప్పుడు కాపీ చేయాల్సిన data తగ్గడం**, అంటే వేగవంతమైన recovery.

### "Consumer వెనకబడితే?"

§7 lo చూసినట్టు, partitions కంటే ఎక్కువ consumers కలపడం సహాయపడదు. కాబట్టి మూడు నిజమైన ఎంపికలు: partitions ఎక్కువ ఉంచడం (§8), ఒక్కో సందేశం యొక్క ప్రాసెసింగ్ ఖర్చు తగ్గించడం, లేదా **ప్రాసెసింగ్ ని consumer నుంచి వేరు చేయడం** – consumer కేవలం ఒక work queue కి తోస్తుంది. కానీ ఆ చివరిది §5 యొక్క క్రమ హామీని రద్దు చేస్తుంది, కాబట్టి అది ఉచితం కాదు.

### "Zombie consumer ని ఎలా ఆపుతారు?"

§12 lo బతికిన మార్పు ఇదే. ప్రతి rebalance కి ఒక **generation number** ఇచ్చి, ప్రతి commit తో దాన్ని పంపడం; broker పాత generation నుంచి వచ్చిన commit ని తిరస్కరిస్తుంది. **ఇది నేను ఈ doc lo అమలు చేయలేదు**, కాబట్టి దీనికి నా దగ్గర సంఖ్య లేదు – కానీ LLD Deep 25 lo ఇదే fencing నమూనాని కొలిచాను: fencing లేకపోతే **2.65% jobs** పాత worker ఫలితాన్ని పట్టుకున్నాయి.

## 16. ఏమి నేర్చుకున్నాం

**1. అందరూ చెప్పే వివరణ పాతబడిపోయి ఉండొచ్చు.** "Sequential disk" ఈ laptop మీద **1.62x**. నిజమైన కారణం batching – **945x**.

**2. fsync bandwidth ని ఖర్చు పెట్టదు, latency ని ఖర్చు పెడుతుంది.** అందుకే సెకనుకి 301, మరియు అందుకే అదే hardware 99,668 machines కి బదులు **106** అవుతుంది.

**3. హారం (denominator) ని జాగ్రత్తగా ఎన్నుకోండి.** "0.4% సందేశాలు" అనేది engineering. "**5.3% keys**" అనేది వ్యాపారం. రెండోది చెప్పండి.

**4. Partition key ఒక performance నిర్ణయం కాదు – అది మీ correctness నిర్వచనం.** "ఏమి క్రమంలో ఉండాలి" అనేదాన్ని మీరు ఆ ఒక్క ఎంపికతో నిర్ణయిస్తారు.

**5. ఒక సంఖ్య మూడు బాధ్యతలు మోస్తే, అది శాశ్వతమైనది.** Partition సంఖ్య సమాంతరతని, key స్థానాన్ని, మరియు అందువల్ల మార్చగలిగే సామర్థ్యాన్ని నిర్ణయిస్తుంది. 16→32 **50.1%** keys ని కదిలిస్తుంది, మరియు ఆ మార్పులో **55.1% keys** క్రమం కోల్పోతాయి.

**6. ఒక setting ఒంటరిగా అబద్ధం చెప్పగలదు.** `acks=all` "అందరికీ" అని కాదు, "ప్రస్తుత ISR lo అందరికీ" అని అర్థం. ISR కుంచిండుకున్న **30.4%** సమయంలో అది మౌనంగా `acks=1` అవుతుంది.

**7. సున్నాకి ఒక ధర ఉంటుంది, మరియు ఆ ధరే అసలు నిర్ణయం.** నష్టం 190 → 0, తిరస్కరణ 0% → **26.6%**.

**8. నియమం తప్పు కావచ్చు, code కాదు.** నా "ఎప్పుడూ క్రమంలోనే" నియమం **400/400** విఫలమైంది – ఎందుకంటే అది క్రమాన్ని మరియు at-least-once ని కలిపేసింది. సరైనది: **మొదటి delivery** క్రమం.

**9. బతికిన mutation కి మూడు అర్థాలు.** బలహీనమైన పరీక్ష, చచ్చిన code, లేదా **harness సృష్టించలేని పరిస్థితి**. ఈసారి మూడోది – మరియు అది చెప్పడం, దాన్ని దాచడం కంటే మెరుగు.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

**Kafka ఇచ్చే హామీలు మేరు అనుకున్నదాని కంటే ఇరుక్కినవి – మరియు అదే వాటిని ఉపయోగకరంగా చేస్తుంది.**

క్రమం topic కి కాదు, **partition** కి. "అన్ని కాపీలు" కాదు, **ప్రస్తుత ISR**. Exactly-once delivery కాదు, **at-least-once + idempotency**.

ప్రతి హామీ ఒక ఇరుక్కిన వాక్యం, మరియు దాని పక్కనే ఒక సంఖ్య ఉంది – 5.3%, 55.1%, 30.4%, 26.6%.

**ఒక system ని అర్థం చేసుకోవడం అంటే అది ఏమి చేస్తుందో తెలుసుకోవడం కాదు – అది ఏమి హామీ ఇవ్వడో ఖచ్చితంగా తెలుసుకోవడం.**